TODO: Paper Title

Anonymous author
Anonymous affiliation

Abstract

Background. TODO: 1–2 sentences on the problem and why it matters.
Aims. TODO: the research questions or specific objectives.
Method. TODO: dataset, models, approaches compared.
Results. TODO: headline numbers, ordered by RQ.
Conclusions. TODO: what this means and the takeaway for practitioners.

2012 ACM Subject Classification Software and its engineering → Software notations and tools; Computing methodologies → Natural language processing

Keywords and phrases issue classification, retrieval-augmented generation, fine-tuning, large language models, mining software repositories

Digital Object Identifier 10.4230/LIPIcs...

1 Introduction

2 Related Work

Early IRC studies employed data mining techniques and ML classifiers [1, 18, 12]. For example, Antoniol et al. [1] leveraged a naive Bayes classifier to distinguish bugs from other issue reports. However, these methods showed low performance due to their limited semantic understanding of the issue report text [13].

To overcome this, recent methods have adopted transformer models due to their strong contextual understanding [6, 4, 24, 16, 17]. These studies typically fine-tune the models on historical issues in a supervised setting. For instance, Izadi et al. [16] fine-tuned a RoBERTa [21] model to classify issues into *Bugs*, *Features*, or *Questions*. Similarly, Trautsch et al. [24] trained a seBERT [25] model to classify issues into the same three categories. Although transformer-based approaches achieve strong results, they rely on large amounts of project-specific data to be effective, hindering their generalizability on projects with a small number of labeled issues [13, 16]. Additionally, they struggle to accurately classify minority classes with fewer training examples [17, 6].

These challenges have motivated the exploration of LLMs, which exhibit strong text classification capabilities thanks to extensive pretraining on massive corpora. [13, 3, 2, 5, 26, 8]. Studies show that LLMs perform well in zero-shot settings [8, 2]. SOTA LLM-based methods achieved the best performance by fine-tuning the models on labeled datasets [13, 3]. For example, Aracena et al. [3] instruction-tuned several models on the NLBSE '23 and '24 datasets.

3 Approach

3.1 Problem Formulation

Given a query issue report, we classify it into only one of the three labels: *bug*, *feature*, or *question*. Only the issue's title and description text are used for classification with no other signals from the issue tracker. The defined label space is adopted directly from prior IRC studies [2, 3, 16, 18, 1].

© Anonymous author(s);
licensed under Creative Commons License CC-BY 4.0
Leibniz International Proceedings in Informatics
LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

3.2 Indexing and Retrieval

To prepare the issues for retrieval based on textual similarity, we first embed their textual content (i.e., *title* + *description*). We do not embed the issue labels. Instead, we retain them as metadata associated with each embedded issue. To generate the embeddings, we use the `all-MiniLM-L6-v2` model from the Sentence-Transformers library [22]. We selected this model due to its lightweight architecture, fast performance, and effectiveness on issue report text [15]. After embedding all issue reports, we index them using the Faiss vector database library [10], which enables efficient high-dimensional similarity retrieval. At the retrieval stage, the query issue is first embedded with the same model. Next, Faiss calculates the cosine similarity of indexed issue reports to the query, and returns the top-K nearest neighbors, ordered by descending similarity.

3.3 RAGTAG

RAGTAG is our proposed retrieval-augmented few-shot prompting approach for IRC. RAGTAG uses the retrieval pipeline described in Section 3.2 to retrieve the top-K nearest neighbors to the query issue, and present them as classified examples to the LLM in a few-shot prompt. This enables a more accurate classification, based on in-context learning [5]. The choice to present the top-K nearest neighbors as examples is based on two insights from prior research: First, retrieval-based example selection outperforms random selection [19]. Second, the most similar neighbors tend to share the same labels [9]; therefore, they are informative for the classification.

We create the few-shot prompt in three parts. First, the system message frames the classification task and the label space. Second, the top-K nearest neighbors are listed in retrieval order (descending similarity), each formatted as *title* + *description* + *label*. Finally, the query issue is appended to the end of the prompt, ensuring it receives high attention from the LLM [20]. K=0 produces a zero-shot prompting baseline, similar to prior work [7].

The LLM is instructed to generate the predicted label wrapped in XML tags (e.g. `<label>bug</label>`). The labels of the presented examples also follow the same format. This technique enables easy detection of the predicted label from the LLM's generation, and reduces variance in LLM outputs caused by prompt format sensitivity [23]. We use a regular expression to extract the predicted label. Unparseable outputs are marked as *invalid*.

3.4 VOTAG

VOTAG is our proposed non-parametric retrieval IRC method. VOTAG uses the retrieval pipeline described in Section 3.2 to retrieve the top-K nearest neighbors of the query issue report, then predicts the final label with a similarity-weighted vote on their labels. This approach follows the standard K-nearest-neighbor classification setup [9], with neighbors weighted by their similarity to the query [11].[1] Formally, for a query issue q with retrieved neighbors $\{(n_i, l_i, s_i)\}_{i=1}^{K}$, where n_i is the i-th nearest neighbor with label l_i and cosine similarity s_i to q, the label is predicted by:

$$\hat{y} = \arg\max_{c \in \mathcal{L}} \sum_{i=1}^{K} s_i \cdot \mathbf{1}[l_i = c], \tag{1}$$

[1] Squared-similarity weighting and uniform majority were also evaluated, but were outperformed by similarity-weighted voting.

where $\mathcal{L}$ is the label set and $\mathbf{1}[\cdot]$ is the indicator function. VOTAG breaks ties by returning the label of the most similar neighbor in the tied classes.

VOTAG has three main roles: First, it serves as a fast and cost-efficient baseline that LLM-based approaches must clear to justify their computational cost. Second, it is used as an ablation to distinguish the contributions of the LLM and retrieval in the RAGTAG method. Third, VOTAG's performance as a function of K informs the K values we evaluate for RAGTAG.

3.5 Fine-Tuning

SOTA IRC approaches have achieved strong performance by supervised fine-tuning of LLMs [13, 3, 2], using Low-Rank Adaptation (LoRA) [14]. In these studies, the model is trained on labeled issue reports and predicts the labels for unseen test issues. Each training example is formatted as an instruction prompt which includes the issue's title and description as input, and its label as output. At inference, the trained model receives the test issue in the same format, and predicts the empty label field. Regular expressions are used to extract the prediction from the output, and unparseable outputs are marked as *invalid*. We use this established methodology as our fine-tuning baseline.

4 Experimental Setup

4.1 Dataset and Settings

We use the dataset introduced by Heo et al. [13] in our experiments. The dataset has a total of 6,600 issue reports gathered from eleven active open-source projects: `ansible/ansible`, `bitcoin/bitcoin`, `dart-lang/sdk`, `dotnet/roslyn`, `facebook/react`, `flutter/flutter`, `kubernetes/kubernetes`, `microsoft/TypeScript`, `microsoft/vscode`, `opencv/opencv`, and `tensorflow/tensorflow`. Each project has 600 issue reports with an equal distribution across the labels (*bug, feature, question*). To support their LLM fine-tuning method, Heo et al. divided the dataset in 50/50 train and test splits, stratified by project and label. This results in 300 train and 300 test issue reports per project.

Heo et al. also evaluate the effect of *project-specific* (PS) and *project-agnostic* (PA) data on fine-tuning with two different configurations.

4.2 Models

4.3 Hyperparameters and Implementation

4.4 Metrics

4.5 Hardware

5 RQ1 – Comparison

6 RQ2 – Diagnosis

7 RQ3 – Bridging

8 Discussion

9 Threats to Validity

10 Conclusion

11 Data Availability

References

1. Giuliano Antoniol, Kamel Ayari, Massimiliano Di Penta, Foutse Khomh, and Yann-Gaël Guéhéneuc. Is it a bug or an enhancement? a text-based approach to classify change requests. In *Proceedings of the 2008 conference of the center for advanced studies on collaborative research: meeting of minds*, pages 304–318, 2008.
2. Gabriel Aracena, Kyle Luster, Fabio Santos, Igor Steinmacher, and Marco A. Gerosa. Applying large language models api to issue classification problem, 2024. URL: `https://arxiv.org/abs/2401.04637`, `arXiv:2401.04637`.
3. Gabriel Aracena, Kyle Luster, Fabio Santos, Igor Steinmacher, and Marco A Gerosa. Applying large language models to issue classification: Revisiting with extended data and new models. *Science of Computer Programming*, page 103333, 2025.
4. Shikhar Bharadwaj and Tushar Kadam. Github issue classification using bert-style models. In *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, pages 40–43, 2022.
5. Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
6. Giuseppe Colavito, Filippo Lanubile, and Nicole Novielli. Issue report classification using pre-trained language models. In *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, pages 29–32, 2022.
7. Giuseppe Colavito, Filippo Lanubile, and Nicole Novielli. Benchmarking large language models for automated labeling: The case of issue report classification. *Information and Software Technology*, page 107758, 2025.
8. Giuseppe Colavito, Filippo Lanubile, Nicole Novielli, and Luigi Quaranta. Leveraging gpt-like llms to automate issue labeling. In *Proceedings of the 21st International Conference on Mining Software Repositories*, pages 469–480, 2024.
9. Thomas Cover and Peter Hart. Nearest neighbor pattern classification. *IEEE transactions on information theory*, 13(1):21–27, 1967.

10 Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. *IEEE Transactions on Big Data*, 2025.

11 Sahibsingh A Dudani. The distance-weighted k-nearest-neighbor rule. *IEEE Transactions on Systems, Man, and Cybernetics*, (4):325–327, 1976.

12 Qiang Fan, Yue Yu, Gang Yin, Tao Wang, and Huaimin Wang. Where is the road for issue reports classification based on text mining? In *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 121–130. IEEE, 2017.

13 Jueun Heo and Seonah Lee. A study on applying large language models to issue classification. In *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*, pages 1–11. IEEE Computer Society, 2025.

14 Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.

15 Huihui Huang, Ratnadira Widyasari, Ting Zhang, Ivana Clairine Irsan, Jieke Shi, Han Wei Ang, Frank Liauw, Eng Lieh Ouh, Lwin Khin Shar, Hong Jin Kang, et al. Back to the basics: Rethinking issue-commit linking with llm-assisted retrieval. *arXiv preprint arXiv:2507.09199*, 2025.

16 Maliheh Izadi. Catiss: An intelligent tool for categorizing issues reports using transformers. In *Proceedings of the 1st international workshop on natural language-based software engineering*, pages 44–47, 2022.

17 Maliheh Izadi, Kiana Akbari, and Abbas Heydarnoori. Predicting the objective and priority of issue reports in software repositories. *Empirical Software Engineering*, 27(2):50, 2022.

18 Rafael Kallis, Andrea Di Sorbo, Gerardo Canfora, and Sebastiano Panichella. Ticket tagger: Machine learning driven issue classification. In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 406–409. IEEE, 2019.

19 Jiachang Liu, Dinghan Shen, Yizhe Zhang, William B Dolan, Lawrence Carin, and Weizhu Chen. What makes good in-context examples for gpt-3? In *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd workshop on knowledge extraction and integration for deep learning architectures*, pages 100–114, 2022.

20 Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173, 2024.

21 Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

22 Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*, 2019.

23 Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. *arXiv preprint arXiv:2310.11324*, 2023.

24 Alexander Trautsch and Steffen Herbold. Predicting issue types with sebert. In *Proceedings of the 1st international workshop on natural language-based software engineering*, pages 37–39, 2022.

25 Julian Von der Mosel, Alexander Trautsch, and Steffen Herbold. On the validity of pre-trained transformers for natural language processing in the software engineering domain. *IEEE Transactions on Software Engineering*, 49(4):1487–1507, 2022.

26 Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021.